Objektorientierte Programmierung

WOCHE 5: QUALITÄTSSICHERUNG – TESTING UND REFACTORING

PETER BERNHARDT

März 2026

1

Teil 1:
Einführung in automatisierte Tests

2

Willkommen zu Woche 5

Software, die ungetestet ist, ist Software, der man nicht vertrauen kann.

Fehler können zu Datenverlust, Sicherheitslücken und Imageschäden führen. Daher ist es essenziell, dass wir unseren Code systematisch testen.

Bisher haben wir unseren Code manuell geprüft, indem wir das Programm gestartet und Eingaben gemacht haben. Das ist auf Dauer nicht tragbar.

Heute lernen wir, wie Code automatisch seinen eigenen Code testet.

3

Die Kosten von unentdeckten Fehlern

Warum testen wir?

Je später ein Fehler im Software-Lifecycle gefunden wird, desto teurer wird seine Behebung.

Die Erkenntnis: Barry Boehm (USC) bewies bereits in den 1980ern, dass Fehler-Kosten exponentiell mit der Zeit steigen.

Die Regel:

- Ein Fehler, der in der **Entwicklung** 1€ kostet (Minuten bis Stunden),
- kostet im **Test** 10€ (Stunden bis Tage),
- und in der **Produktion** 100€ (Krisenmeetings, Datenverlust, massiver Imageschaden)

Die Konsequenz: Investitionen in Qualität (Tests, Reviews, Architektur) zahlen sich vielfach aus.

4

Was ist ein Unit Test?

Ein **Unit Test** ist ein kleines, automatisiertes Code-Snippet, das eine spezifische Funktionalität der Anwendung (meist eine einzige Methode einer Klasse) prüft.

Er stellt sicher, dass sich die Methode exakt so verhält, wie der Entwickler es bei der Erstellung vorgesehen hat.

Ein Unit Test kommt mit einem **Test-Framework** (z.B. xUnit) und einer **Assertion-Bibliothek** (z.B. Assert) daher, um die Ergebnisse zu überprüfen.

Ein Unit Test kann beliebig oft ausgeführt werden, ohne dass menschliches Eingreifen nötig ist.

5

Ziele des Unit Testings

1. **Korrektheit:** Beweisen, dass der Code tut, was er soll.
2. **Regressionsschutz:** Sicherstellen, dass neue Änderungen alte Funktionen nicht brechen.
3. **Dokumentation:** Tests zeigen anderen Entwicklern, wie eine Klasse benutzt werden soll.
4. **Design-Check:** Code, der schwer zu testen ist, ist meist auch schlecht entworfen (enge Kopplung).

Das letztendliche Ziel ist nicht, unbedingt 100% Test-abdeckung zu erreichen, sondern die kritischen Pfade und Logiken abzusichern.

6

Teil 2: Tests schreiben mit xUnit

7

xUnit in .NET

`xUnit` ist das De-facto-Standard-Framework für das Testen von C#-Anwendungen.

Entwickelt von den ursprünglichen Machern von `NUnit`, bietet es eine moderne, flexible und leichtgewichtige API.

Es ist Open Source, aktiv gepflegt und tief in die .NET-Ökosystem integriert.

Es bietet uns Attribute wie `[Fact]` und `[Theory]`, um dem Compiler mitzuteilen: “Dies ist keine normale Methode, sondern ein Testlauf.”

8

Die Struktur: Das AAA-Pattern

Jeder gute Unit Test folgt dem AAA-Schema:

1. **Arrange (Vorbereiten):** Testobjekte instanziieren, Dummy-Daten anlegen.
2. **Act (Ausführen):** Genau *eine* Methode am Testobjekt aufrufen.
3. **Assert (Prüfen):** Das Ergebnis der Methode mit dem erwarteten Ergebnis vergleichen.

Ziel: Klarheit und Fokus. Jeder Test sollte nur eine Sache testen, damit er leicht verständlich und wartbar bleibt.

9

Der Unit Testing Lifecycle

Ein Testlauf besteht aus:

1. **Setup:** (Optional) Vorbereiten der Umgebung.
2. **Execution:** Ausführen des Tests (AAA).
3. **Teardown:** (Optional) Aufräumen (z.B. Dateien löschen).
4. **Reporting:** Das Test-Framework zeigt an, ob der Test bestanden oder fehlgeschlagen ist.

In `xUnit` wird für jeden Test eine neue Instanz der Testklasse erstellt, was für maximale Wiederholbarkeit sorgt.

10

xUnit Attribute: [Fact]

Das **[Fact]** Attribut kennzeichnet einen invarianten (unveränderlichen) Testfall.

Also eine Prüfung, die immer das gleiche Ergebnis liefern sollte, egal welche Daten vorliegen (innerhalb des Szenarios).

```
[Fact]
public void Validieren_SollteTrueZurueckgeben_WennId11StelligIst()
{
    // Arrange
    var validator = new SteuerIdValidator();
    string korrekteId = "12345678901";

    // Act
    bool result = validator.Validieren(korrekteId);

    // Assert
    Assert.True(result);
}
```

11

Einschub: Die Wichtigkeit der Namensgebung bei Tests

Testnamen müssen nicht kurz sein.

Sie sollten wie eine Dokumentation gelesen werden können.

Testmethoden sollten so benannt werden, dass das getestete Verhalten, die Bedingung und das erwartete Ergebnis unmittelbar erkennbar sind. Dadurch ist auch nach längerer Zeit schnell verständlich, was der Test prüft.

```
Muster:     [Methode]_[ErwartetesErgebnis]_Wenn[Szenario]

Beispiel 1: Validieren_GibtTrueZurueck_WennId11StelligIst()
Beispiel 2: BerechneGebuehr_GibtRabattZurueck_WennFahrzeugElektroIst()
Beispiel 3: Registriere_WirftException_WennBuergerUnter18Ist()
```

Wenn der Test in der Entwicklungs-Pipeline fehlschlägt, weiß der Entwickler anhand des Namens schnell, wo etwas "kaputt" ist.

12

xUnit Attribute: [Theory]

Mit **[Theory]** kennzeichnen wir parametrisierte Tests.
Wir sagen: "Diese Logik gilt für eine ganze Reihe von Daten."

```
[Theory]
public void Validieren_SollteFalseZurueckgeben_WennIdUngueltigIst
                                                    (string ungueltigeId)
{
    // Arrange
    var validator = new SteuerIdValidator();

    // Act
    bool result = validator.Validieren(ungueltigeId);

    // Assert
    Assert.False(result);
}
```

Aber wie übergeben wir die Testdaten?

13

xUnit Attribute: [InlineData] (1/2)

[InlineData] ist der einfachste Weg, Testdaten direkt im Code zu definieren. Sie können so viele `[InlineData]`-Attribute hinzufügen, wie Sie möchten, um verschiedene Szenarien abzudecken.

```
// Test-Methode mit einem Parameter

[Theory]
[InlineData("123")] // Zu kurz
[InlineData("123456789012")] // Zu lang
[InlineData("ABC45678901")] // Buchstaben
public void Validieren_SollteFalseZurueckgeben_WennIdUngueltigIst
                                                    (string ungueltigeId)
{
    var validator = new SteuerIdValidator();
    bool result = validator.Validieren(ungueltigeId);
    Assert.False(result);
}
```

14

xUnit Attribute: [InlineData] (2/2)

[InlineData] ist der einfachste Weg, Testdaten direkt im Code zu definieren. Sie können so viele `[InlineData]`-Attribute hinzufügen, wie Sie möchten, um verschiedene Szenarien abzudecken.

```
// Test-Methode mit drei Parametern inklusive erwartetem Ergebnis

[Theory]
[InlineData(1, 1, 2)]
[InlineData(2, 2, 4)]
public void Addieren_GibtSummeZurueck_WennEingabenGueltigSind
                                        (int a, int b, int erwartet)
{
    var addierer = new Addierer();
    decimal ergebnis = addierer.Addieren(a, b);
    Assert.Equal(erwartet, ergebnis);
}
```

15

Assertions: Mehr als nur True/False

Die **Assert**-Klasse bietet viele hilfreiche Methoden:

```
Assert.Equal(expected, actual)

Assert.True(condition)

Assert.Contains("Teiltext", resultString)

Assert.Throws<ArgumentException>(() => method())

Assert.Empty(collection)
```

Diese Methoden ermöglichen es, sehr präzise zu prüfen, ob das Ergebnis den Erwartungen entspricht.

16

Teil 3:
Mocking von Abhängigkeiten

17

Die wichtigste Test-Regel: Isolation

Ein Unit Test muss *isoliert* ablaufen.

Er darf nicht auf das Dateisystem, nicht auf das Netzwerk und vor allem nicht auf eine echte Datenbank zugreifen!

Warum?

Weil der Test sonst unzuverlässig ("flaky") wird.

Es ginge nicht mehr nur darum, ob Ihr Code korrekt ist, sondern auch ob die Infrastruktur gerade mitspielt (eingeschränkte Wiederholbarkeit).

18

Das Problem: Infrastruktur-Abhängigkeiten

Stellen Sie sich vor, Sie wollen den `RegistrierungsService` testen.

Dieser Service speichert Daten in einer echten Datenbank und sendet eine echte E-Mails.

Problem:

Wenn die Datenbank beim Test *down* ist, schlägt Ihr Test fehl, obwohl Ihr Code korrekt ist.

Der Test ist z.B. langsam und hinterlässt “Müll” in der DB.

19

Die Lösung: Warum Interfaces?

Weil wir gegen Interfaces programmieren, können wir die “echte” Datenbank im Test durch eine Attrappe (Mock) ersetzen.

Ein Mock implementiert das Interface, tut aber nichts Kritisches.

Es simuliert nur das Verhalten, das für den Test nötig ist (z.B. Zähler hochzählen, um zu prüfen, ob die Methode aufgerufen wurde).

Zur Erstellung eines Mocks gibt es zwei Möglichkeiten:

1. **Manuell:** Sie schreiben eine Klasse, die das Interface implementiert und die Methoden so anpasst, dass sie nur das tun, was für den Test nötig ist (z.B. Zähler hochzählen).
2. **Mocking-Frameworks:** Es gibt Bibliotheken wie `Moq`, die es ermöglichen, Mocks dynamisch zu erstellen und Verhalten zu definieren, ohne eine konkrete Klasse schreiben zu müssen.

20

Der zu testende Service (Code)

Dieser Service soll getestet werden. Er hängt vom Interface `IDatenbank` ab:

```
public class RegistrierungsService
{
    private readonly IDatenbank _db;
    public RegistrierungsService(IDatenbank db) => _db = db;

    public void Registriere(Buerger b)
    {
        if (b.Alter < 18) throw new ArgumentException("Zu jung");
        _db.Speichern(b);
    }
}
```

Hier wollen wir die Logik testen, dass Bürger unter 18 Jahren nicht registriert werden dürfen.

Wir wollen aber nicht, dass unser Test von der Datenbank abhängig ist.

21

Der Mock-Einsatz im Test

Im Test-Projekt bauen wir uns also manuell eine Attrappe, die so tut, als wäre sie die Datenbank.

```
public class DatenbankMock : Idatenbank {
    public int SpeichernAufrufe { get; private set; } = 0;

    public void Speichern(Buerger b) {
        SpeichernAufrufe++; // Wir zählen nur mit, um zu sehen,
                            // ob Speichern aufgerufen wurde.
    }
}

// Im Arrange-Teil des Tests nutzen wir den Mock:
var dbMock = new DatenbankMock();
var service = new RegistrierungsService(dbMock);
```

So testen wir **nur** die Logik des Services, ohne eine „echte“ Datenbank zu benötigen.

22

Nutzung von Mocking-Frameworks

Mit einem Framework wie **Moq** können wir den Mock viel schneller erstellen:

```
// Arrange - Wir nutzen nun Moq:
var dbMock = new Mock<IDatenbank>();
var service = new RegistrierungsService(dbMock.Object);

// Act – Test mit Bürger > 18 Jahre; fehlt der Kürze halber…

// Assert - Prüfen, dass Speichern genau 1x aufgerufen wurde.
// Wenn die Verifikation nicht passt, wirft Moq eine MockException
// und der Test schlägt fehl.
dbMock.Verify(db => db.Speichern(It.IsAny<Buerger>()), Times.Once);
```

Insbesondere bei komplexen Interfaces oder vielen Methoden spart das Mocking-Framework viel Zeit und Code.

23

Teil 4:
Test-Driven Development (TDD)

24

Was ist Test-Driven Development? (1/2)

TDD dreht den klassischen Workflow um.

Wir schreiben den Test *nicht* am Ende, wenn das Feature fertig ist.

Wir schreiben den Test **zuerst**, bevor überhaupt eine einzige Zeile Fachcode existiert!

25

Was ist Test-Driven Development? (2/2)

TDD ist eine Disziplin, die zu besserem Design, höherer Qualität und mehr Vertrauen in den Code führt.

Es zwingt uns, über die Anforderungen und das Design nachzudenken, bevor wir mit dem Coden beginnen.

Nicht jeder Test muss mit TDD geschrieben werden, aber die wichtigsten und kritischsten Logiken sollten es sein.

Nachteilig ist, dass es anfangs ungewohnt und zeitintensiv sein kann.

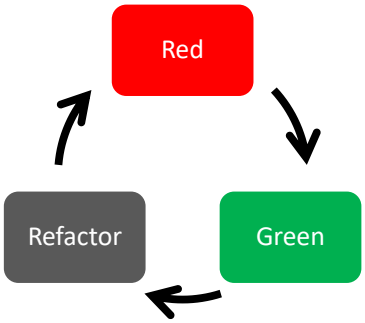
Es erfordert Übung, um den Workflow zu meistern.

26

Der Rot-Grün-Refactor Zyklus

TDD ist ein iterativer Zyklus, der in drei Phasen abläuft:

1. **Red (Rot):**
   Einen fehlschlagenden Test schreiben.
2. **Green (Grün):**
   Den minimal nötigen Code schreiben, um den Test grün zu machen.
3. **Refactor (Aufräumen):**
   Den entstandenen Code elegant und sauber strukturieren.



Dieser Zyklus wird so lange wiederholt, bis die Funktionalität vollständig implementiert ist.

27

Phase 1: RED

- Sie überlegen sich anhand der Spezifikation, was die zu implementierende Methode tun soll.
- Sie schreiben den Test, der diese Funktionalität prüft.
- Da die Klasse oder Methode noch gar nicht existiert, kompiliert es am Anfang vielleicht nicht mal.
  Das ist in Ordnung.
- Schreiben Sie eine leere Hülle für die Methoden. Sie müssen den Test nur zum Laufen bringen, damit er fehlschlägt.
- Sie führen den Test aus: Er ist ROT.

Warum ist das wichtig?
So stellen Sie sicher, dass Ihr Test überhaupt in der Lage ist, fehlzu-schlagen (und nicht durch einen Designfehler immer “grün” anzeigt).

28

Phase 2: GREEN

- Sie schreiben den Produktiv-Code.
- Wichtig: Schreiben/nutzen Sie keine komplexen Architekturen oder Design-Patterns.
- Schreiben Sie nur das, was nötig ist. Es ist völlig in Ordnung, wenn der Code “hässlich” oder “unfertig” ist. Hauptsache, der Test besteht, d.h. die Funktionalität ist da.
- Das Ziel ist einzig und allein, den Test so schnell wie möglich grün (erfolgreich) zu bekommen.

Warum ist das wichtig?
Weil Sie so schnell Feedback bekommen, ob Ihre Implementierung korrekt ist. Sie vermeiden es, Zeit in die falsche Richtung zu investieren.

29

Phase 3: REFACTOR

- Der Test ist grün. Sie haben nun ein Sicherheitsnetz!
- Jetzt schauen Sie sich den Produktiv-Code an.
- Gibt es Magic Numbers? Ist die Methode zu lang? Sind die Variablennamen schlecht? Ist die Klasse zu groß? Gibt es Duplikate? Sollten Design-Patterns angewendet werden?
- Sie passen den Code an.
- Nach **jeder** kleinen Änderung lassen Sie die Tests laufen. Solange sie grün bleiben, haben Sie nichts kaputt gemacht.

Warum ist das wichtig?
Weil es so leicht ist, in der “Green”-Phase unstrukturierten Code zu produzieren. Refactoring sorgt dafür, dass Ihr Code sauber und wartbar bleibt, ohne die Funktionalität zu gefährden.

30

Warum TDD so mächtig ist

Vorteile:

- **100% Testabdeckung:** Da kein Code ohne vorherigen Test geschrieben wird, ist das gesamte System automatisch getestet.
- **Lebende Dokumentation:** Die Tests beschreiben das Systemverhalten besser als jedes Word-Dokument.
- **Besseres Design:** Wer TDD anwendet, baut automatisch entkoppelte, gut strukturierte Klassen (da sie sonst gar nicht vorab testbar wären).

31

Warum TDD keine „Allzweckwaffe“ ist

Nachteile:

- Anfangs **ungewohnt** und (immer) **zeitintensiv** (meint: teuer).
- Ohne eine gute **Spezifikation** kann es schwierig sein, die Tests zu schreiben.
- Es erfordert Disziplin, um den **Workflow** zu meistern.

Kompromiss: Nicht jeder Test muss mit TDD geschrieben werden, aber die wichtigsten und kritischsten Logiken sollten es sein.

32

Teil 5: Refactoring-Techniken

33

Was ist Refactoring?

Refactoring ist die Verbesserung der inneren Struktur ohne Änderung des äußeren Verhaltens.

Es ist das Aufräumen des Codes, um ihn lesbar und wartbar zu halten.

Refactoring ist ein kontinuierlicher Prozess, der während der gesamten Softwareentwicklung stattfindet.

Es ist kein einmaliges Ereignis, sondern eine Praxis, die in den Alltag eines Entwicklers integriert werden sollte.

34

Warum Refactoring?

Code der während der Entwicklung nicht regelmäßig refaktorisiert wird, neigt dazu, unübersichtlich und schwer wartbar zu werden.

Das führt zu “Code Smells” (z.B. lange Methoden, duplizierter Code, unklare Namen), die das Risiko von Fehlern erhöhen und die Entwicklung neuer Features erschweren.

Der Code „verrottet“.

35

Häufige Refactoring-Techniken (1/2)

Die folgende Liste ist nicht vollständig, aber sie deckt einige der häufigsten und effektivsten Refactoring-Techniken ab:

1. **Extract Method:** Eine zu lange Methode in mehrere kleine aufteilen.
2. **Rename Variable:** Unklare Namen (z.B. `x`) durch sprechende Namen (z.B. `antragsId`) ersetzen.
3. **Move Method:** Eine Methode in die Klasse verschieben, zu der sie logisch gehört.
4. **Remove Duplication:** Den “DRY” (Don’t Repeat Yourself) Grundsatz anwenden.

36

Häufige Refactoring-Techniken (2/2)

Die folgende Liste ist nicht vollständig, aber sie deckt einige der häufigsten und effektivsten Refactoring-Techniken ab:

5. **Introduce Parameter Object:** Wenn eine Methode zu viele Parameter hat, diese in ein Objekt packen.
6. **Replace Magic Numbers:** Unklare Zahlen durch benannte Konstanten ersetzen.
7. **Encapsulate Field:** Direkten Zugriff auf Felder durch Getter/Setter ersetzen.

Refactoring ist kein „Hexenwerk“!

37

Wann refaktorisieren?

- Wenn Sie eine neue Funktion hinzufügen wollen, aber der bestehende Code zu starr ist (Refactor first!).
- Wenn Sie “Code Smells” entdecken.
- Wenn Sie die Lesbarkeit verbessern wollen.
- Wenn Sie die Wartbarkeit erhöhen wollen.

„Pfadfinder-Regel":
Always leave the code better than you found it.

38

Die goldene Regel des Refactorings

Niemals refaktorisieren, wenn die Unit-Tests rot sind!

Ohne ein grünes Sicherheitsnetz ist Refactoring extrem riskant und führt oft zu neuen Fehlern, die erst spät bemerkt werden.

Daher ist es essenziell, dass Sie vor jedem Refactoring sicherstellen, dass alle Tests grün sind.

So können Sie sofort erkennen, wenn Sie etwas "kaputt" gemacht haben.

39

Teil 6: Zusammenfassung

40

Wrap-up Woche 5

Unit Tests: Sichern die Basis ab und dienen als lebendige Dokumentation.

xUnit & AAA: Ein strukturierter Standard für alle .NET Entwickler.

Mocking: Isoliert Logik von unzuverlässiger Infrastruktur.

TDD: Ein Workflow, der zu besserem Design und hoher Qualität führt, jedoch seinen Preis hat.

41

Ausblick auf nächste Woche

In der finalen Theorie-Woche (Woche 6) betrachten wir asynchrone Programmierung (`async`/`await`) und professionelles Error Handling.

Zudem besprechen wir die Details zu Ihren Abschlussprojekten!

42